

Ex.No.10 Use Ghidra to disassemble and analyze the malware code.

Project Objectives

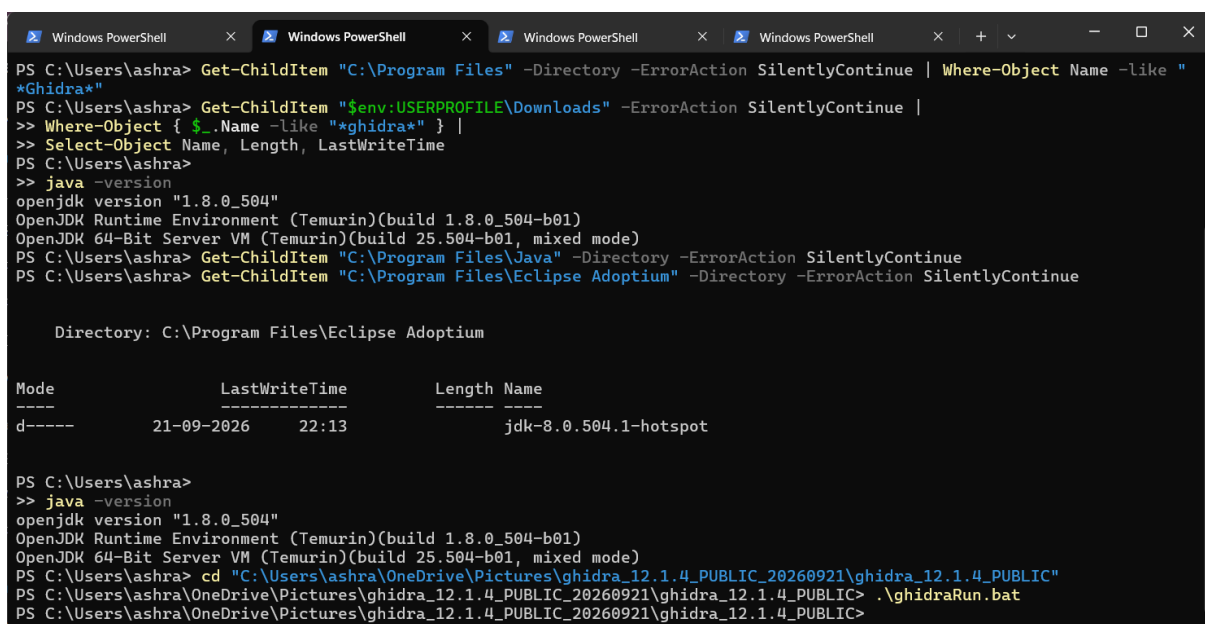
- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
- To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
- To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- **Overview:** Brief description of Ghidra, its installation, and how it is used for malware analysis.
- **Requirements:** List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- **Project Outline:** A breakdown of the tutorial steps, linking to various sections or files within the repository.



```
PS C:\Users\ashra> Get-ChildItem "C:\Program Files" -Directory -ErrorAction SilentlyContinue | Where-Object Name -like "*Ghidra*"
PS C:\Users\ashra> Get-ChildItem "$env:USERPROFILE\Downloads" -ErrorAction SilentlyContinue |
>> Where-Object { $_.Name -like "*ghidra*" } |
>> Select-Object Name, Length, LastWriteTime
PS C:\Users\ashra>
>> java -version
openjdk version "1.8.0_504"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_504-b01)
OpenJDK 64-Bit Server VM (Temurin)(build 25.504-b01, mixed mode)
PS C:\Users\ashra> Get-ChildItem "C:\Program Files\Java" -Directory -ErrorAction SilentlyContinue
PS C:\Users\ashra> Get-ChildItem "C:\Program Files\Eclipse Adoptium" -Directory -ErrorAction SilentlyContinue

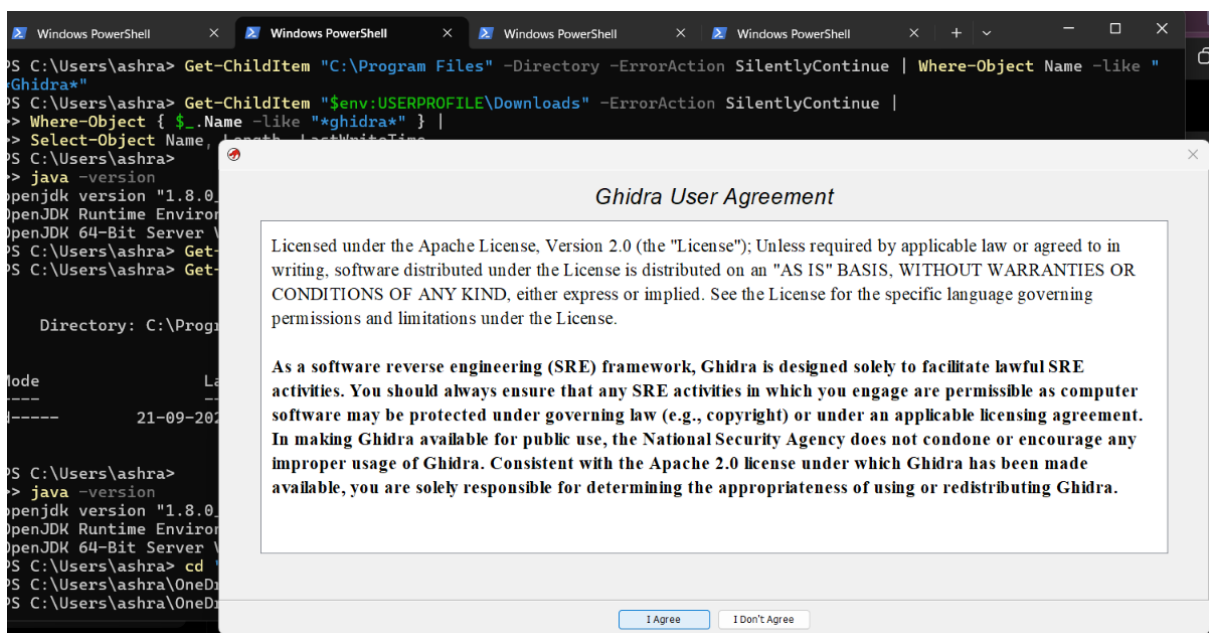
Directory: C:\Program Files\Eclipse Adoptium

Mode                LastWriteTime         Length Name
----                -
d-----          21-09-2026    22:13          jdk-8.0.504.1-hotspot

PS C:\Users\ashra>
>> java -version
openjdk version "1.8.0_504"
OpenJDK Runtime Environment (Temurin)(build 1.8.0_504-b01)
OpenJDK 64-Bit Server VM (Temurin)(build 25.504-b01, mixed mode)
PS C:\Users\ashra> cd "C:\Users\ashra\OneDrive\Pictures\ghidra_12.1.4_PUBLIC_20260921\ghidra_12.1.4_PUBLIC"
PS C:\Users\ashra\OneDrive\Pictures\ghidra_12.1.4_PUBLIC_20260921\ghidra_12.1.4_PUBLIC> .\ghidraRun.bat
PS C:\Users\ashra\OneDrive\Pictures\ghidra_12.1.4_PUBLIC_20260921\ghidra_12.1.4_PUBLIC>
```

2. Tutorial Steps (Markdown or Jupyter Notebooks)

- **Step-by-step instructions, broken down into digestible sections, such as:**
 - **Environment Setup:** Instructions on setting up a virtualized and isolated environment.
 - **Initial Analysis:** How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - **Function Analysis:** Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - **String and Import Analysis:** How to locate and interpret strings and imports related to malicious behavior.
 - **Advanced Techniques:** Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.



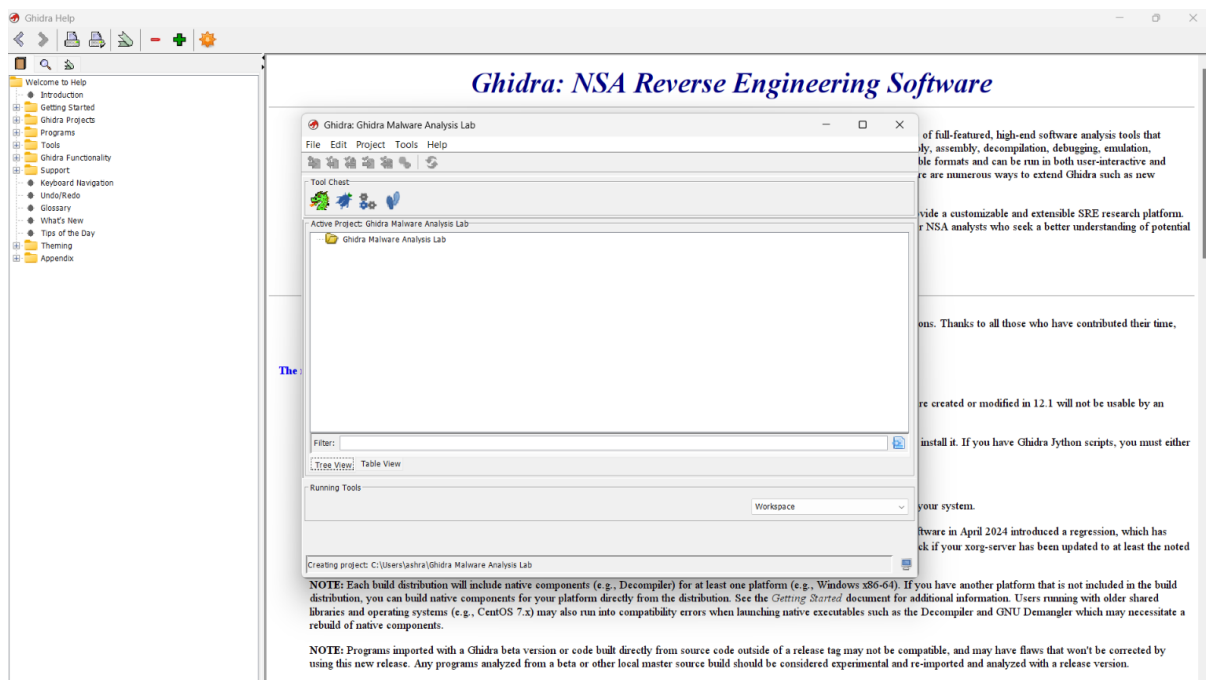
3. Example Scripts (Python or Java)

- **Scripts to automate repetitive analysis tasks, such as:**
 - **Extracting and listing all string references.**
 - **Identifying network-related functions or common indicators of malware.**
 - **Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).**

- **Script Examples:** Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).

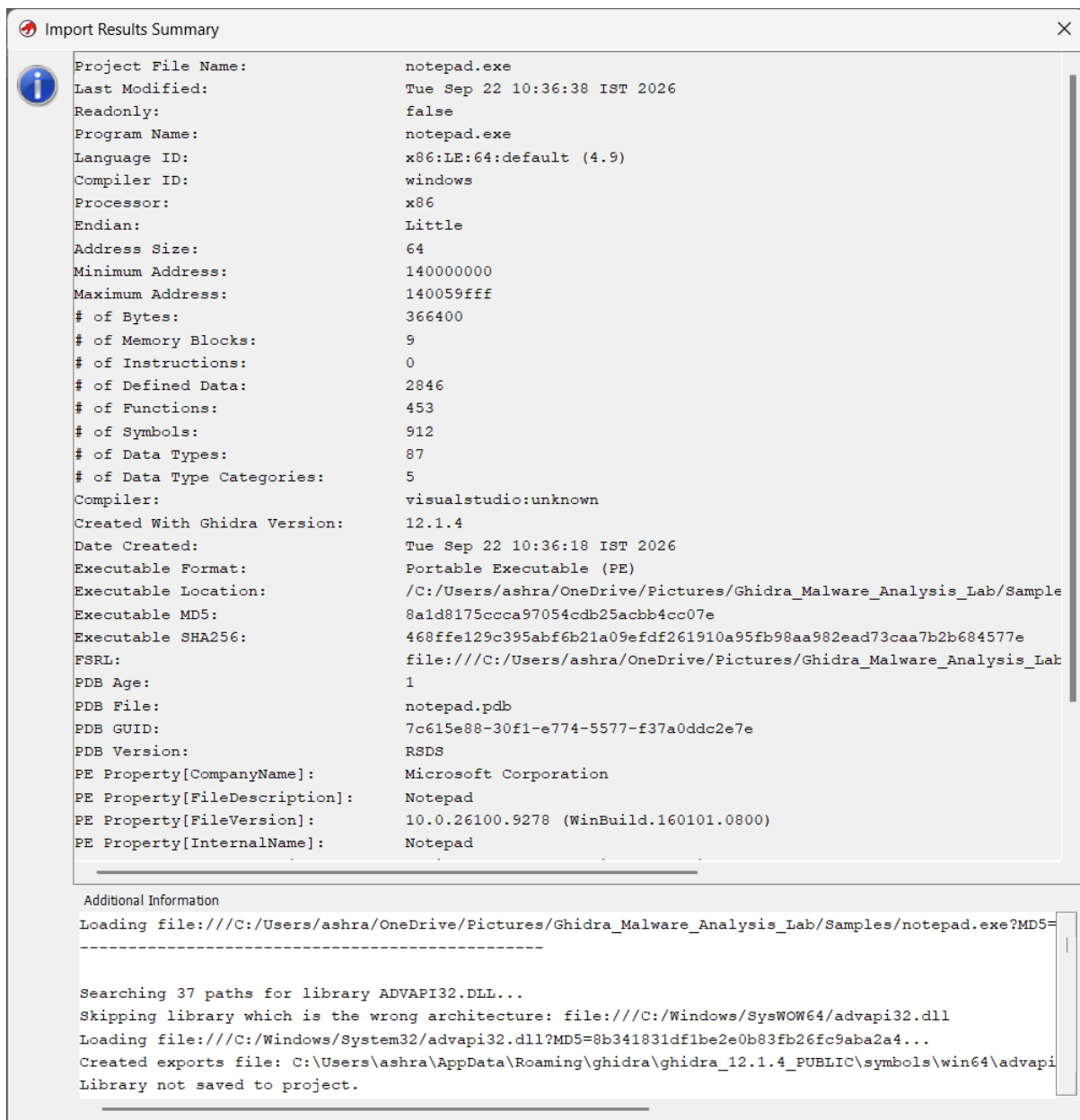
4. Sample Data (Benign Binary or Hex Dump)

- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
- Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
- Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.



5. Documentation and Comments

- Include explanations of the code within each script, detailing what each function does.
- Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.



6. Report Template

- **A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:**
 - **Summary of Analysis:** Brief description of the suspected malware.
 - **Function Analysis:** Summary of the key functions, their purpose, and the findings.
 - **Behavioral Indicators:** Summary of malware behaviors, such as persistence, file system manipulation, or network communications.

-
- The screenshot shows the Ghidra Malware Analysis Lab interface. The main window displays the assembly code for a function, with a decompiler view on the right showing 'No Function'. The left sidebar contains the Program Tree, Symbol Tree, and Data Type Manager. The bottom status bar indicates the current address is 1400019C0.
- Program Tree:**
- notepad.exe
 - Headers
 - .text
 - .forlik
 - .pdata
 - .data
 - .pdata
 - .dldat
 - .rsrc
 - .reloc
- Symbol Tree:**
- Imports
 - Exports
 - Functions
 - Labels
 - Classes
 - Namespaces
- Data Type Manager:**
- Data Types
 - Builtin Types
 - notepad.exe
 - windows_n12_64
- Assembly Code:**
- ```

1400019b5 00 77 00h
1400019b6 00 77 00h
1400019b7 00 77 00h
1400019b8 00 77 00h
1400019b9 00 77 00h
1400019ba 00 77 00h
1400019bb 00 77 00h
1400019bc 00 77 00h
1400019bd 00 77 00h
1400019be 00 77 00h
1400019bf 00 77 00h

+ FUNCTION *****
+
undefined entry()
 assume SE_OFFSET = 0xffff0000000
 <UNASSIGNED> <RETURN>
entry XREF[4]: Entry Point(*), 140002a5c0(*), 140002a5c0(*), 140002a5c0(*)
1400019c0 48 83 ec 20 SUB RSP,0x20
1400019c4 e8 cf 05 CALL FUN_140001f98
00 00
1400019c9 48 83 c4 20 ADD RSP,0x20
1400019cd e9 6e fe JMP FUN_140001040
ff ff

-- Flow Override: CALL_RETURN (CALL_TERMINATOR)
1400019d2 00 77 00h
1400019d3 00 77 00h
1400019d4 00 77 00h

```
- Decompiler:**
- ```

1: No Function
  
```
- Console - Scripting:**
- 1400019C0 entry undefined (1)
- Decompiling FUN_140002384